

Analisi e Risoluzione di Problemi Algoritmici

MATTEO MEROLLI

matteomerollim1@gmail.com

Corso di Algoritmi e Problem Solving

Docente: *Prof. Matteo Cavaliere*

Università degli Studi di Modena e Reggio Emilia (FIM)

Indice

1	BrainStorming	1
1.1	Descrizione esercizio risolto	1
1.2	Descrizione Soluzione A	1
1.3	Descrizione Soluzione B	1
1.4	Misura empirica tempi	2
1.5	Formato Input e Output	2
2	Battery Usage	4
2.1	Descrizione esercizio risolto	4
2.2	Descrizione Soluzione A	4
2.3	Descrizione Soluzione B	4
2.4	Misura empirica tempi	5
2.5	Formato Input e Output	5
3	Working Breaks	7
3.1	Descrizione esercizio risolto	7
3.2	Descrizione Soluzione A	7
3.3	Descrizione Soluzione B	7
3.4	Misura empirica tempi	8
3.5	Formato Input e Output	8
4	TrainStations	9
4.1	Descrizione esercizio risolto	9
4.2	Descrizione Soluzione A	9
4.3	Descrizione Soluzione B	9
4.4	Misura empirica tempi	10
4.5	Formato Input e Output	11
5	Dream Team	12
5.1	Descrizione esercizio risolto	12
5.2	Descrizione Soluzione A	12
5.3	Descrizione Soluzione B	12
5.4	Misura empirica tempi	13
5.5	Formato Input e Output	14
6	IA Declaration	15

1 BrainStorming

1.1 Descrizione esercizio risolto

In una sessione di brainstorming delle persone propongono nomi. In particolare il partecipante 0 propone un nome qualsiasi mentre tutti gli altri (partecipanti numero i) devono proporre un nome tale che: la prima lettera del nome i -esimo coincida con l'ultima lettera del nome $i - 1$ -esimo e che per ogni nome i -esimo sia diverso da tutti gli i nomi precedenti.

La sessione finisce quando gli N partecipanti hanno proposto ciascuno un nome. Il programma deve leggere il numero di partecipanti (N) e poi i nomi relativi proposti in sequenza. Deve inoltre avvisare se nessun nome rompe i vincoli, altrimenti indica la posizione del primo vincolo.

1.2 Descrizione Soluzione A

La soluzione A usa la soluzione più naturalmente umana, cioè per ogni nome proposto si verifica se mantiene il vincolo dell'ultimo carattere del precedente nome coincidente con il primo carattere del nuovo nome proposto. Qualora questo test fosse verificato correttamente si procede a verificare se l'attuale nome ricevuto non sia già stato presente nell'elenco. Qualora non fosse presente, viene aggiunto. Se anche solo 1 dei test citati non fosse verificato il programma procede ad avvisare ed interrompersi.

Costo: $\mathcal{O}(N^2)$, visto che le stringhe sono N e per ciascuna si verificano le precedenti ad essa, si fanno $\frac{N(N-1)}{2}$ confronti. Ma in caso di errori (non rispetto dei vincoli) potrebbe concludersi in meno visto che il programma al primo errore si arresta.

1.3 Descrizione Soluzione B

La soluzione B invece usa un approccio più tecnico, in particolare si appoggia sul concetto di mappe hash, in questo caso a due livelli. Ad ogni stringa vengono associati due hash, il primo che ne determina la sua posizione nell'array in cui vengono mappati i nomi, e il secondo lo identifica nella sua lista bucket (che dipende dal primo hash). Qualora l'hash uno coincida e ci sia un elemento in tale lista bucket con stesso hash due, allora è abbastanza probabile che siano uguali, quindi si procede con il confronto fra stringhe. Tutto questo è implementato usando l'hash OAAT con mod diversi. La lettura, controllo del vincolo su lettera iniziale del nuovo nome uguale a ultima nome precedente e infine notifica d'errore rimangono le medesime.

Costo: $\mathcal{O}(N^2)$ nel caso peggiore (tutti gli elementi hanno stessi hash1), ma vista l'ottima scelta di funzione hash e di mod (numeri grandi primi) possiamo garantire che la probabilità è pressoché nulla. Anzi, salvando solo l'hash 2 e non la stringa intera per verificare l'uguaglianza si avevano risultati corretti garantiti (empiricamente) fino ad $N = 10.000$. Oltre, effettivamente creava qualche collisione, quindi il costo mediamente si aggira al lineare $\mathcal{O}(N)$.

1.4 Misura empirica tempi

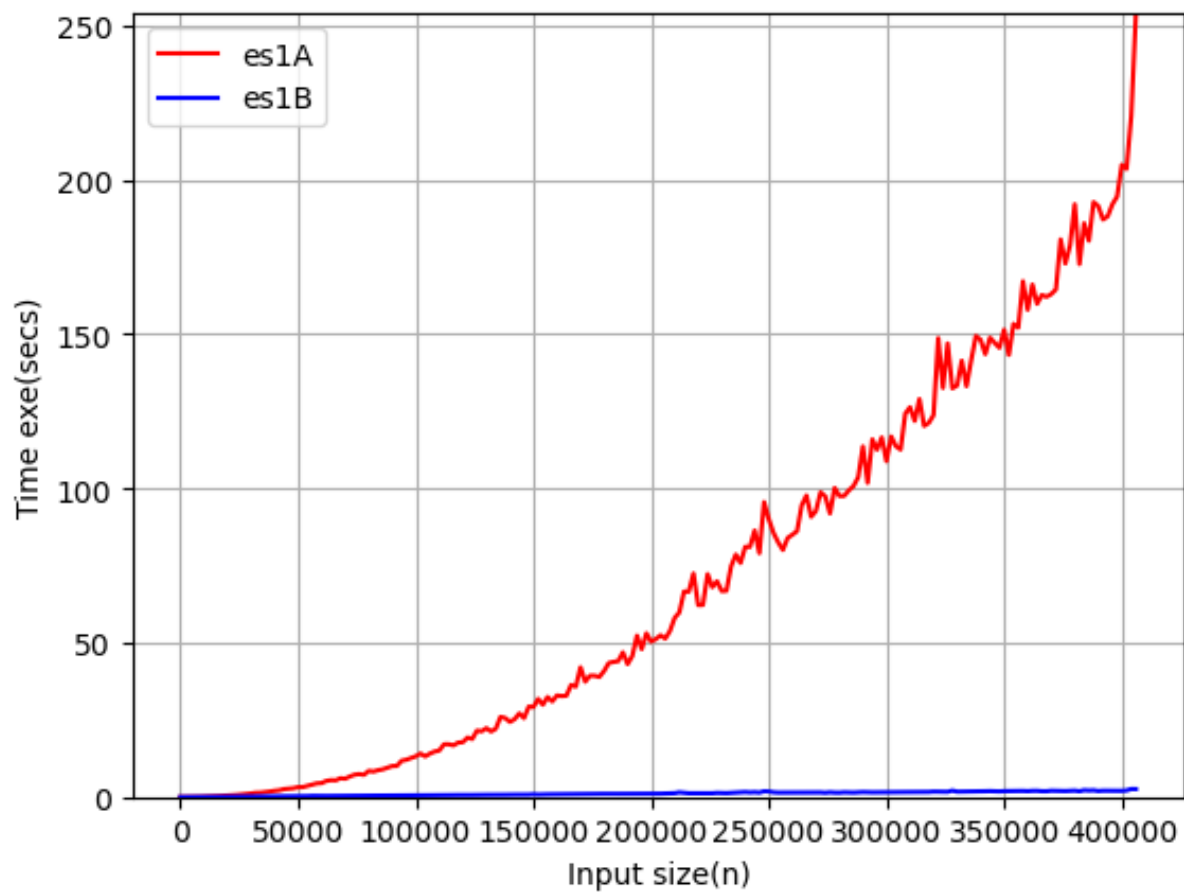


Figura 1: Confronto prestazioni ES1A vs ES1B

1.5 Formato Input e Output

Input:

```
N (Numero di nomi)
nome1
nome2
...
nomeN
```

Output: Se è tutto corretto → nice brainstorm! Se non è tutto corretto → Error in {numero di primo nome sbagliato} name

Esempio 1:

```
Input:
3
algorithm
nvidia
anthropic
```

```
Output:
Error in 2 name
```

Esempio 2:

```
Input:
```

```
3
programcounter
rust
thread

Output:
nice brainstorm!
```

Esempio 3:

```
Input:
4
hash
heap
push
hash

Output:
Error in 4 name
```



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

2 Battery Usage

2.1 Descrizione esercizio risolto

Un robot di servizio dispone di una quantità intera di unità di batteria detta T , può svolgere tre compiti diversi: A, B e C ciascuno con un costo in unità di batteria intero che verranno comunicati come parte dell'input. Si vuole determinare e restituire la scelta dei compiti in modo da minimizzare la batteria inutilizzata e fra quelle con batteria inutilizzata minima quella che massimizza il numero totale di compiti eseguiti.

Il programma deve restituire quindi il numero massimo di compiti eseguiti, la batteria non utilizzata qualora ci fosse e i compiti eseguiti per raggiungere tale risultato.

2.2 Descrizione Soluzione A

La Soluzione A adotta un approccio puramente ricorsivo di tipo top-down. L'algoritmo esplora tutte le possibili opzioni a partire dal problema principale, suddividendolo in sottoproblemi via via più piccoli. Ad esempio, dato un tempo totale a disposizione T , l'algoritmo valuta ogni singola scelta possibile sottraendo ricorsivamente il rispettivo costo temporale (es. $T = T - a$, $T = T - b$, ecc.). Questo processo continua fino al raggiungimento del caso base, ovvero quando il tempo residuo non consente ulteriori attività. Infine, i risultati delle diverse ramificazioni vengono confrontati per estrarre la sequenza ottimale.

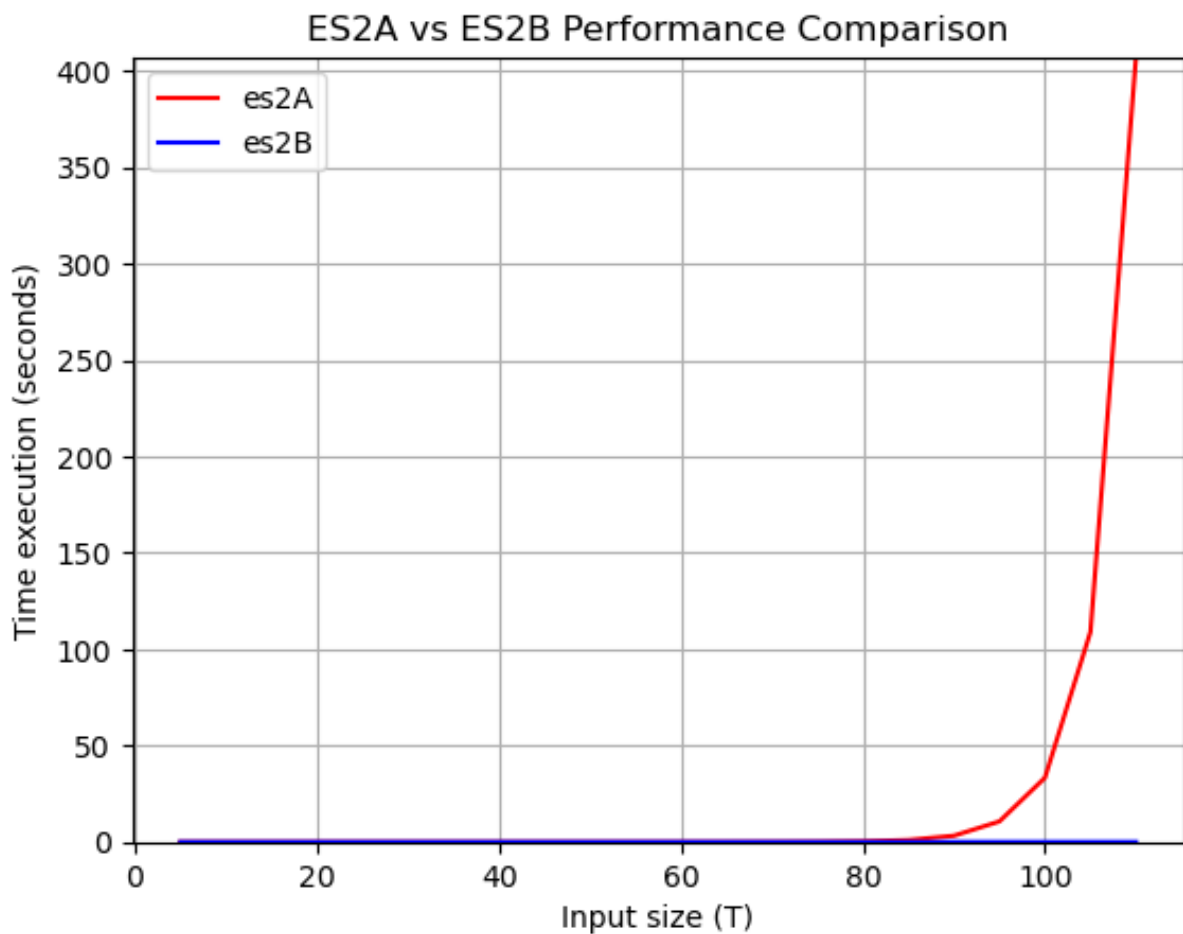
Costo: $O(3^T)$, poiché ogni T (tempo) chiama ricorsivamente $T - a/b/c$. Nel caso peggiore chiama tutti e tre, in altre parole stiamo percorrendo un albero decisionale con 3 possibili scelte ad ogni nodo che notoriamente ha questo tipo di costo.

2.3 Descrizione Soluzione B

La Soluzione B adotta un approccio basato sulla programmazione dinamica (dynamic programming). Questa tecnica consiste nel risolvere i sottoproblemi in ordine crescente di dimensione, memorizzandone i risultati per costruire in modo efficiente la soluzione del problema principale. Ciò è reso possibile dal fatto che il problema gode della proprietà di sottostruttura ottima (pur non ammettendo l'approccio greedy): la soluzione ottima globale può essere ricavata a partire dalle soluzioni ottime dei sottoproblemi di dimensione inferiore, combinandole con un numero di operazioni di ordine costante $O(1)$. L'impiego della programmazione dinamica migliora drasticamente l'efficienza dell'algoritmo: memorizzando i risultati intermedi (attraverso la tabulazione o la memoization), si evita di ricalcolare il valore della funzione per gli stati già visitati, valutando ciascun sottoproblema una sola volta.

Costo: $O(T)$. L'algoritmo risolve un massimo di T sottoproblemi e, per ciascuno di essi, esegue un numero costante di operazioni per determinare la scelta ottima.

2.4 Misura empirica tempi



11/5

Figura 2: ES2A vs ES2B Performance Comparison

2.5 Formato Input e Output

Input:

```
Tempo a disposizione  
Tempo di A, B, C
```

Output: Se è possibile fare qualcosa → **remaining T:** {T-UsedTime} + le scelte fatte
Se non possibile fare qualcosa → **nothing to do, IMP**

Esempio 1:

```
Input :  
10  
2 3 4  
  
Output :  
remaining T: 0  
Optimal is done via choosing:  
a 5 times  
b 0 times  
c 0 times
```

Esempio 2:

```
Input:
1
4 5 6

Output:
nothing to do, IMP
```

Esempio 3:

```
Input:
17
2 3 5

Output:
remaining T: 0
Optimal is done via choosing:
a 7 times
b 1 times
c 0 times
```



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

3 Working Breaks

3.1 Descrizione esercizio risolto

In una realtà lavorativa è necessario che ogni tecnico effettui una pausa obbligatoria durante il suo turno di lavoro, ma non possono essere in pausa più di due lavoratori allo stesso tempo. Il turno di lavoro è detto T (intero), la realtà lavorativa è composta da N lavoratori, ciascuno con pausa necessaria di k_i minuti con i che va da 1 a N . Si vuole dunque “schedulare” le pause affinché siano fatte in modo da garantire il corretto funzionamento aziendale.

3.2 Descrizione Soluzione A

La soluzione A per risolvere il problema, utilizza un approccio di brute force implementato mediante la percorrenza di un albero decisionale in cui tramite la ricorsione si esplorano tutti i possibili scenari. In particolare, a partire dal tempo=0 (cioè l’inizio) l’algoritmo esplora tutte le scelte possibili una per una ed esplora fino al caso base (tempo=tempo a disposizione totale) e lì verifica se le scelte verificano i vincoli (in particolare che tutti abbiano fatto la propria pausa nel turno di lavoro), dal momento in cui il programma rileva che ci sono più di 2 lavoratori in pausa, abbandona il ramo senza esplorarlo ulteriormente (pruning).

Costo: $O(N^{2T})$ ad ogni istante temporale (nodo dell’albero decisionale) vengono considerate tutte le possibili scelte di al massimo due lavoratori tra gli N disponibili. Il numero di scelte per ciascun nodo dell’albero è quindi $O(N^2)$, e poiché queste scelte si ripetono per T livelli della ricorsione, il numero totale di configurazioni esplorate cresce come una moltiplicazione su T livelli, portando alla complessità computazionale menzionata.

3.3 Descrizione Soluzione B

La soluzione B invece cambia completamente l’idea del problema, cioè si visualizza il problema come un problema dello zaino (risolto chiaramente tramite dynamic programming), in cui si cerca un insieme dei lavoratori che minimizzi il tempo di lavoro senza nessuno in pausa, creando così due “pile”(imbuti, non LIFO) cioè quella appena creata e i lavoratori rimanenti. Si verifica poi che anche la seconda pila abbia somma delle pause necessarie minore di T .

Se questo controllo risulta vero, allora si ha che il problema ha soluzione, cioè fin dal primo istante T si toglie un elemento dalla pila di lavoratori che effettua la sua pausa finché finisce dando posto al prossimo nella pila, visto che il numero massimo di pile sono 2 (per definizione) e nelle pile un solo lavoratore è in pausa allora siamo sicuri che i vincoli del problema siano sempre rispettati.

Se questo controllo risulta falso allora si può affermare che non esiste soluzione.

Costo: $O(N \cdot T)$, con T = tempo, N = numero lavoratori, la parte più “costosa” è dunque la soluzione del problema dello zaino che implementato via dynamic programming ha notoriamente questo costo.

3.4 Misura empirica tempi

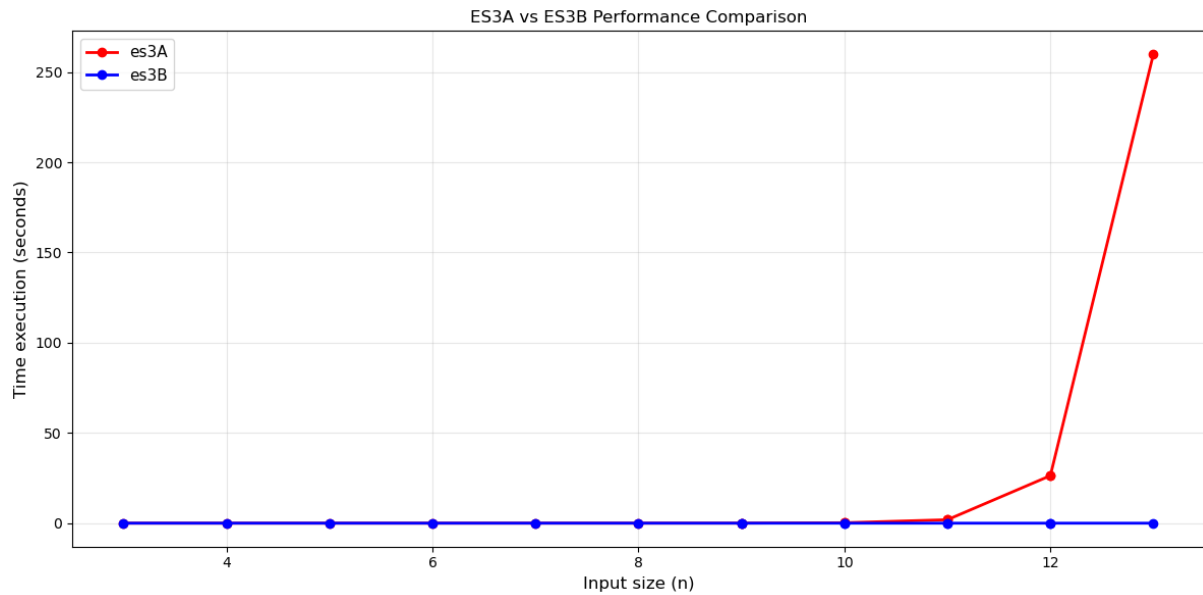


Figura 3: ES3A vs ES3B Performance Comparison

3.5 Formato Input e Output

Input:

```
Tempo Numero lavoratori
pausa di lavoratore 1
pausa di lavoratore 2
...
```

Output: Se è possibile fare qualcosa → quando ogni lavoratore i -esimo dovrebbe iniziare la pausa in una sol.

Se non possibile fare qualcosa → no possible sol

Esempio 1:

```
Input:
3 1
1
```

```
Output:
2
```

Esempio 2:

```
Input:
2 3
1 1 1
```

```
Output:
no possible sol
```

Esempio 3:

```
Input:
4 3
2 2 1
```

```
Output:
0 0 3
```

4 TrainStations

4.1 Descrizione esercizio risolto

Esiste una rete ferroviaria in linea retta lunga L metri e ci sono N stazioni intermedie note, si vuole: massimizzare la distanza minima fra due fermate in tutta la linea ferroviaria, tenendo conto che la prima, l'ultima e una di controllo (dettata dall'istanza d'input del problema) non si possono togliere, e togliendo al massimo M stazioni.

4.2 Descrizione Soluzione A

La soluzione A per risolvere il problema, utilizza un approccio di brute force implementato mediante la percorrenza di un albero decisionale e backtracking per effettuare tutte le scelte possibili. In particolare, per ogni stazione si cerca di non toglierla, poi di toglierla e quando si arriva al caso base (si è presso la stazione finale) si verifica di quanto è la spaziatura minima e si mantiene l'insieme delle scelte fatte per la migliore fino a quel momento. Una volta tentate tutte le scelte possibili è ovvio che avremmo sicuramente incontrato e mantenuto l'ottimale.

Costo: $\mathcal{O}(L + 2^N N)$ si contano il numero di chiamate ricorsive (albero decisionale con 2 possibili scelte per ogni stazione, ciascuna con propria verifica correttezza (fare N volte al massimo un'operazione di ricerca prossima stazione che è $\mathcal{O}(1)$) + L ritenuto degno di essere inserito nel costo visto che in istanze in cui la linea fosse estremamente lunga ($1M$), il costo dello scorrimento di elementi (fatto di 64 in 64) può superare il fattore decisione (costo relativo a N).

4.3 Descrizione Soluzione B

La soluzione B invece cambia completamente approccio per uno più tecnico, dunque si effettua la ricerca della soluzione ottimale dividendo il problema in due problemi, uno di feasibility di una possibile soluzione e uno della sua ottimizzazione tramite binary search. Si procede quindi, a cercare il limite dello spazio delle soluzioni, poiché la fattibilità è monotona rispetto alla distanza (se è fattibile D , lo sono tutti i valori minori; se non è fattibile D , non lo sono nemmeno i valori maggiori), la ricerca binaria permette di individuare in modo esatto e rapido il valore soglia, ovvero la distanza massima possibile. L'unico drawback di questa soluzione è che per ogni verifica di una possibile soluzione, si copia la bitmap contenente la configurazione delle stazioni attuali, questo spreco di memoria è contenuto dall'utilizzo stesso di una bitmap che dunque richiede poco spazio per istanze contenenti anche molti elementi.

Costo: $\mathcal{O}(L + M \cdot \log_2(L))$, dunque la parte dominante è la ricerca binaria con estremi 0 e L che tutte le volte richiama una funzione di verifica feasibility di costo M . A questo però si aggiunge un costo lineare di $L/64$ per la allocazione tramite `calloc` della bitmap.

4.4 Misura empirica tempi

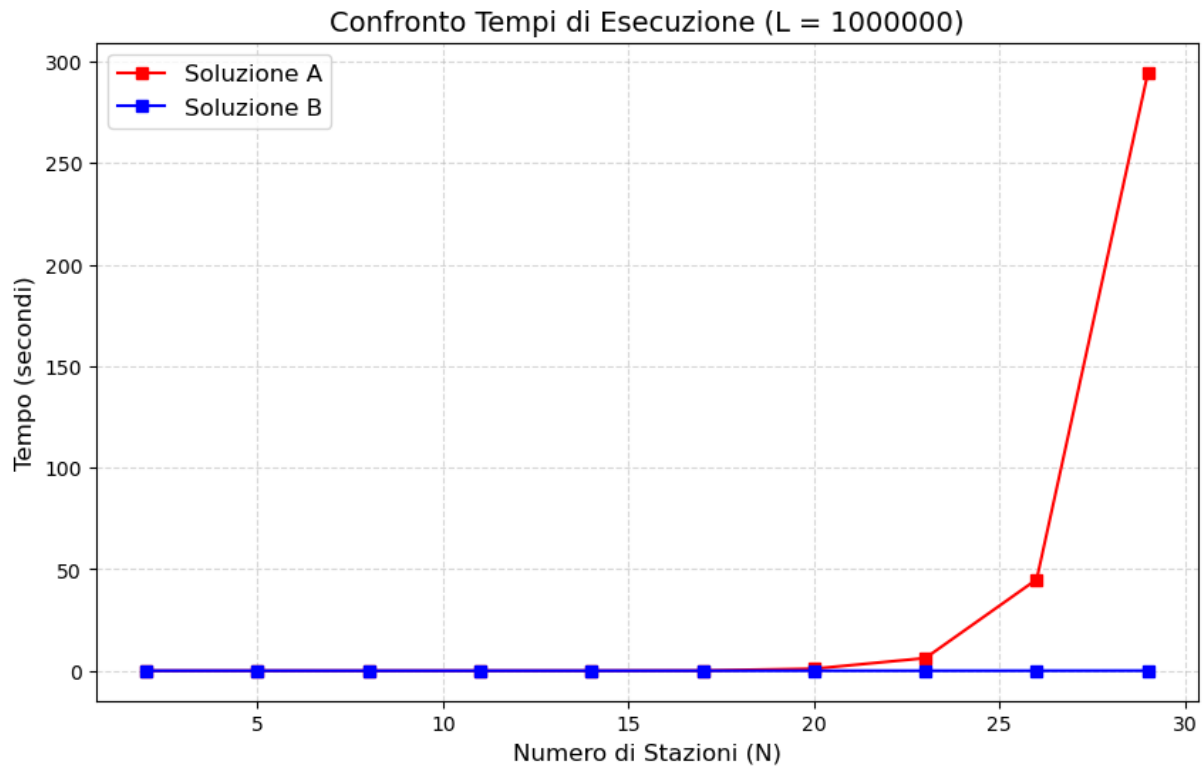


Figura 4: ES4A vs ES4B Performance Comparison

Approfondimento sul grafico dell'efficienza di B per dati enormi per conferma che l'overhead della copia della bitmap non è nociva al tempo di esecuzione, inoltre conferma la crescita lineare/superlineare.

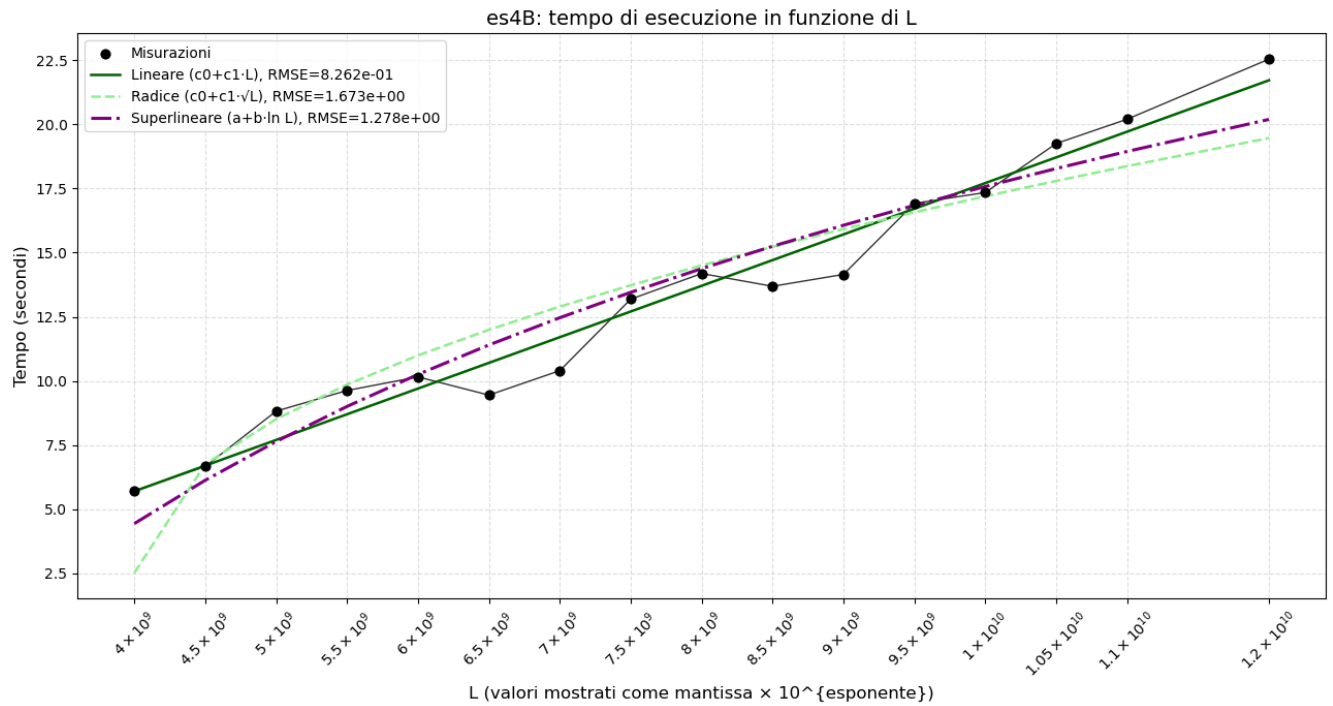


Figura 5: test megalomani su es4B

4.5 Formato Input e Output

Input:

```
L, N, M, posizione stazione di controllo
posizione stazione 1
posizione stazione 2
...
```

Output: Massimo valore spaziatura: valore Stazioni chiuse: indici stazioni chiuse

Esempio 1:

```
Input:
20 3 1 10
4 8 16

Output:
Massimo valore spaziatura: 4
Stazioni chiuse: 8
```

Esempio 2:

```
Input:
10 1 0 10
5

Output:
Massimo valore spaziatura: 5
Stazioni chiuse: 0
```

5 Dream Team

5.1 Descrizione esercizio risolto

Si hanno a disposizione un insieme di candidati (di cardinalità C) che sono caratterizzati da ID, affidabilità e criticità, si vuole formare la squadra migliore possibile di P candidati, la forza di una squadra è uguale a: $\min(\text{affidabilità}) - \max(\text{criticità})$.

5.2 Descrizione Soluzione A

La Soluzione A adotta un approccio randomizzato basato sul metodo Monte Carlo. Invece di esplorare in modo esaustivo tutto lo spazio delle soluzioni, l'algoritmo genera un numero calcolato di combinazioni casuali di squadre e tiene traccia di quella con la qualità massima.

Il numero di tentativi N non è inserito a mano, ma viene calcolato dinamicamente dalla funzione `calcola_tentativi` per garantire matematicamente una confidenza del 99,99% di aver testato la squadra ottima.

Calcolo della probabilità: Il numero totale di combinazioni possibili è dato dal coefficiente binomiale $\binom{C}{P}$. La probabilità di fallimento singolo (ovvero la probabilità di non pescare casualmente la combinazione ottima in una singola estrazione) è:

$$P_{\text{fallimento_singolo}} = 1 - \frac{1}{\binom{C}{P}}$$

Eseguendo N tentativi indipendenti, la probabilità di fallimento totale diventa:

$$P_{\text{fallimento_totale}} = \left(1 - \frac{1}{\binom{C}{P}}\right)^N$$

L'algoritmo esegue un ciclo while aumentando N di uno fino a che la probabilità di successo $(1 - P_{\text{fallimento_totale}})$ non supera il target prefissato (0.9999). In realtà il numero di tentativi è stato limitato a 10^8 per permettere al programma di effettivamente concludere anche nei casi peggiori (aumentando probabilità di errore).

Costo: $\mathcal{O}\left(P \cdot \binom{C}{P}\right)$: dunque si tentano circa le combinazioni di C oggetti in P posti volte un'operazione di costo P (valuta team).

5.3 Descrizione Soluzione B

La seconda soluzione sfrutta invece la programmazione dinamica per ottenere il risultato esatto dell'ottimo globale senza dover verificare un numero straordinario di possibili squadre.

Come prima operazione si ordinano tutti i candidati in base alla loro affidabilità, disponendoli in ordine decrescente dal valore più alto a quello più basso. Successivamente, si applica la programmazione dinamica per minimizzare la criticità massima k , concentrandosi sulla costruzione di sottogruppi composti al massimo da $P - 1$ persone. Una volta completata la matrice, si esegue un ciclo finale in cui si prova ad aggiungere l' i -esimo elemento a ciascuna delle migliori combinazioni precedentemente salvate in `dp[P-1][i-1]`. Questo passaggio è fondamentale: poiché i candidati sono stati ordinati in modo decrescente, l'ultimo elemento inserito sarà matematicamente quello con l'affidabilità minore, andando così a fissare il valore di affidabilità minima dell'intero team completo. A questo punto, si calcola la qualità complessiva combinando i dati appena uniti, si tiene traccia del record massimo trovato e, infine, si effettua il backtracking sulla matrice per ricostruire a ritroso gli ID dei componenti scelti per la squadra.

Dunque l'ordinamento iniziale ci permette di focalizzare la matrice DP esclusivamente sulla sottominimizzazione delle criticità.

Definizioni e ricorrenza della Programmazione Dinamica:

- **Sottoproblemi:** Minimizzare la criticità massima scegliendo esattamente p membri (con $p < P$) valutando un sottoinsieme formato solo dai primi i candidati disponibili.
- **Stato della matrice:** $dp[p][i]$ rappresenta la minima criticità massima ottenibile formando un gruppo di esattamente p persone scelte tra i primi i candidati.
- **Equazione di Bellman:**

$$dp[p][i] = \min(dp[p][i-1], \max(dp[p-1][i-1], K_i))$$

- **Esclusione del candidato i:** $dp[p][i-1] \rightarrow$ non prendo il candidato, quindi mantengo la miglior criticità calcolata senza di lui per p persone.
- **Inclusione del candidato i:** $\max(dp[p-1][i-1], K_i) \rightarrow$ includo il candidato nel team. La nuova criticità massima diventa il valore maggiore tra la sua criticità personale (K_i) e la criticità ottimale del gruppo precedente di $p-1$ persone.

Costo: $O(C \log C + C \cdot P)$: i costi maggiori sono l'ordinamento di tutti i candidati in base alla affidabilità e il riempimento della tabella della programmazione dinamica.

5.4 Misura empirica tempi

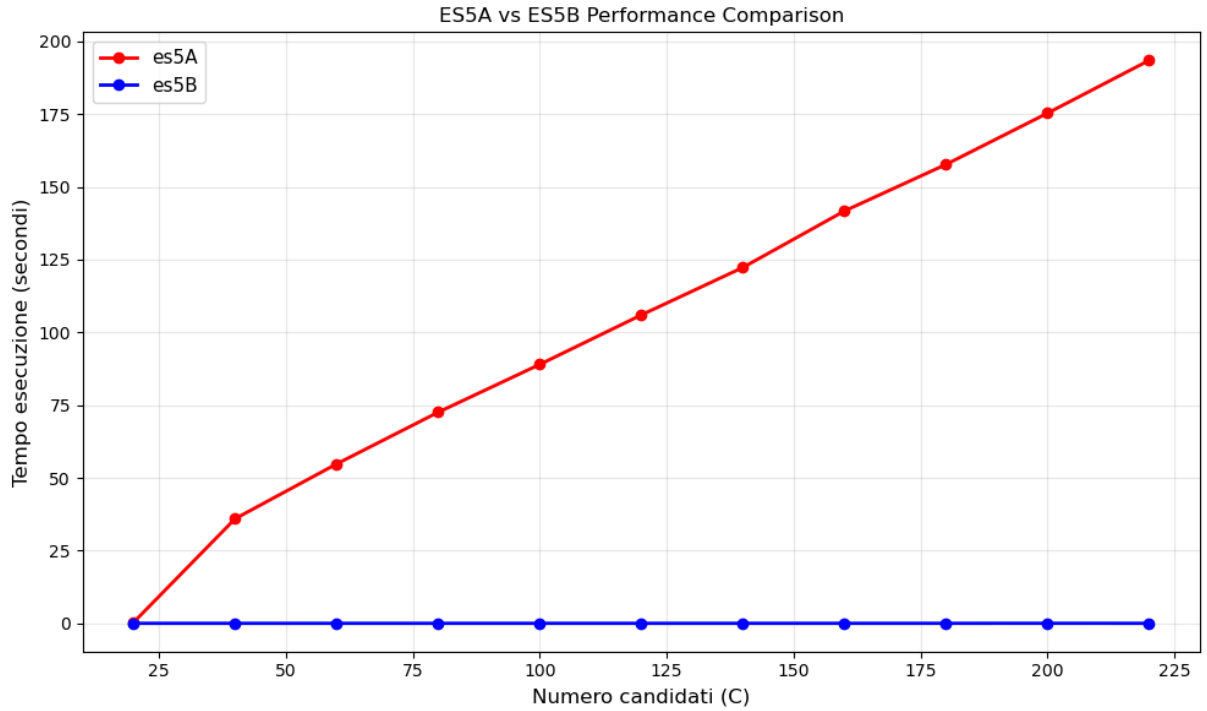


Figura 6: ES5A vs ES5B Performance Comparison

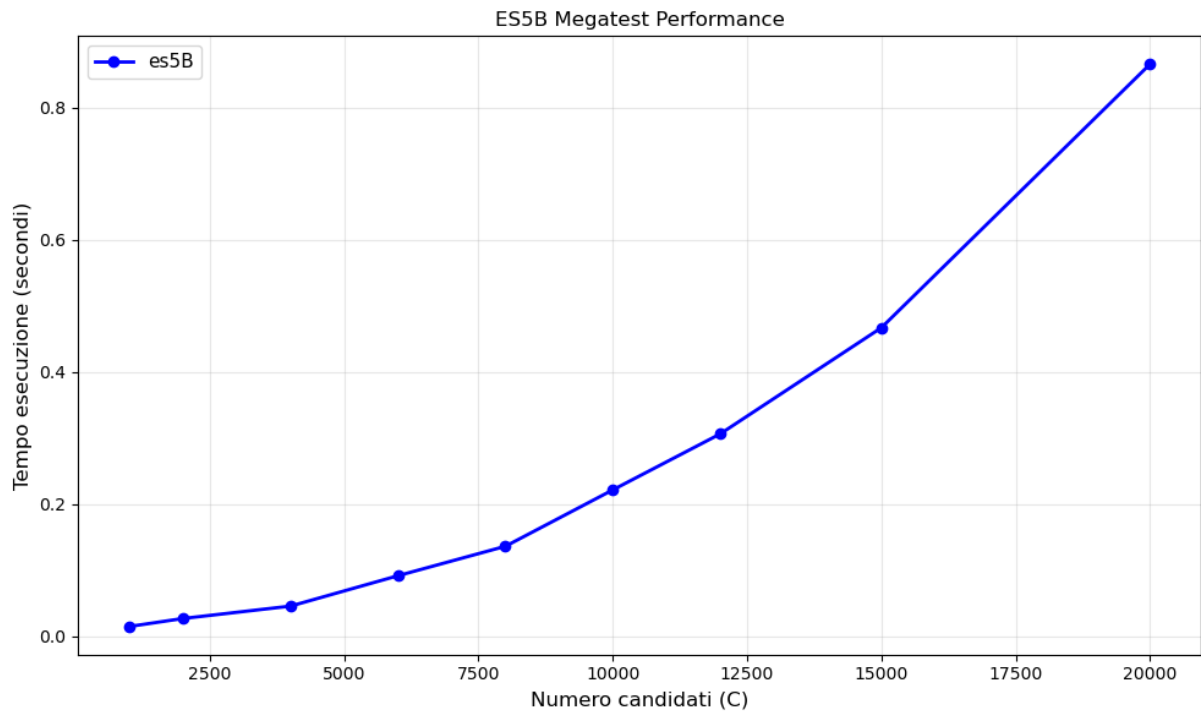


Figura 7: test megalomani su es5B, il costo di es5A per istanze simili è troppo elevato per poter essere anch'esso incluso, il numero di tentativi da fare soffre di esplosione combinatoria

5.5 Formato Input e Output

Input:

```
Numero candidati(C); Numero candidati da selezionare (P)
affidabilita' lavoratore i; criticita' lavoratore i
i=(1...C)
```

Output: Se è possibile formare una squadra → Valore della migliore squadra di P persone e l'elenco degli id da scegliere Se non possibile formare una squadra → no possible sol

Esempio 1:

Input:

```
4 2
10 5
8 2
6 1
4 8
```

Output:

```
Qualita' massima del team: 4
Componenti del team scelti (ID): 2 3
```

Esempio 2:

Input:

```
3 5
10 2
9 3
8 4
```

Output:

```
no possible sol
```

6 IA Declaration

Durante lo svolgimento del corso è stato fatto uso di intelligenza artificiale laddove segnalato nel codice. In particolare è stata utilizzata per generare alcune funzioni tediose già note, generare test ed eventualmente per aiutarmi a comprendere alcune tecniche/errori fatti durante le esercitazioni in classe.

In particolare:

Es1:

La funzione: `int hash_oaat(const char *str, const int mod)`

Es2:

Le funzioni: `int choose_max(int a, int b, int c, int *value)`
restituisce 1 se il max è a, 2 se è b e 3 se è c, mette il valore massimo in value.
`int get_abc(int A, int B, int C, int whatihavechosen)`
Mappa whatihavechosen in 1 2 3 come spiegato precedentemente.

Es3:

Le funzioni: `int countCurrentInPause(int pauses[], int t, int n, int *whenpausestarts)`
Restituisce il numero di lavoratori in pausa con le scelte attuali al tempo t.
`int knapsack_fill(int pauses[], int T, int n, int *prev, int *picked)`
Risolve un problema dello zaino senza ripetizione in cui i pesi sono pauses (cioè la pausa che deve fare ogni lavoratore) e la capacità è T.
`int *buildAstack(int n, int prev[], int picked[], int found)`
Esegue il backtracking della soluzione. Restituisce un array allocato dinamicamente che funge da bitmap, dove il valore 1 all'indice *i* indica che l'i-esimo lavoratore è assegnato al turno della "pila A".

Es4:

Le primitive di bitmap in C e `int findNextStation(uint64_t *bm, int current, int L)`
[= nota implementazione che cerca il prossimo uno in una bitmap in tempo costante]

Es5:

`int calcola_tentativi(int C, int P, long double target)`
`void genera_team_random(int C, int P, int choices[])`
`long double permutations(int C, int P)`
La ricostruzione tramite backtracking della sol di es5B